

Bài 6

CÁC THUẬT TOÁN SẮP XẾP

IV. SẮP XẾP KIỂU CHÈN(*INSERT Sort*)

Thuật toán này dùng trong trường hợp tạo một danh sách, vừa đưa phần tử vào vừa so sánh lựa chọn vị trí đặt phần tử theo đúng trật tự yêu cầu (tăng dần hay giảm dần giá trị). Khi nhập vào số mới, phải so sánh với những số đã nhập trước đó, nếu nó lớn hơn số đã nhập cuối cùng thì chỉ việc đưa vào vị trí sau số đã nhập, nếu không phải so sánh hết với các số trước đó.

Ví dụ, nhập lần lượt các số vào từ bàn phím, mỗi lần nhập số mới cần so sánh với các số đã nhập để sắp xếp sao cho dãy đã nhập có giá trị tăng dần.

Cụ thể : khai báo dãy số $x[5]$;

Bước 1. cho $i=1$, nhập $x[1] = 67$

67				
----	--	--	--	--

Bước 2. $i=2$;

Thêm biến phụ r để lưu số mới nhập; nhập $r = 33$

So sánh số mới nhập 33 với số 67 trước đó, vì $33 < 67$ nên đưa số 33 ra trước số 67.

Đoạn thuật toán của việc “So sánh số mới nhập 33 với số 67 trước đó, vì $33 < 67$ nên đưa số 33 ra trước số 67” được diễn đạt như sau :

Thêm biến j để chỉ vị trí phần tử đứng trước phần tử mới nhập, $j=i-1=1$, $x[j]=67$.

So sánh $r(=33)$ với $x[j]$, nếu $(r < x[j])$ là TRUE thì $x[j+1] = x[j]$; $x[j] = r$; nếu FALSE thì $x[j+1] = r$.

Câu lệnh if là : $\text{if}(r < x[j]) \{x[j+1]=x[j]; x[j]=r\}$ else $x[j+1]=r$;

Kết quả :

33	67			
----	----	--	--	--

Bước 3. Tăng $i=3$; nhập số mới $r = 21$; $j=i-1$; ($j=2$), $x[j]=67$.

Để sắp xếp đúng trật tự tăng dần, cần so sánh số 21 với 67, đưa 21 về trước 67:

Thuật toán của việc này được diễn đạt là :

So sánh $r(=21)$ với $x[j](=67)$, nếu $(r < x[j])$ là TRUE thì $x[j+1] = x[j]$; $x[j] = r$; nếu FALSE thì $x[j+1] = r$. Kết quả :

33	21	67		
----	----	----	--	--

Vì $r = 21$ còn đứng sau 33 nên cần phải sắp xếp tiếp để đưa về vị trí đúng trật tự tăng dần, hiện tại r đang ở vị trí $x[2]$.

Để làm tiếp, phải giảm $j=j-1$; (j mới $=1$), $x[j] = 33$.

Lặp lại :

So sánh $r(=21)$ với $x[j](= 33)$, nếu $(r < x[j])$ là TRUE thì $x[j+1] = x[j]$; $x[j] = r$; nếu FALSE thì $x[j+1] = r$

Kết quả :

21	33	67		
----	----	----	--	--

Bước này đến đây phải dừng, nghĩa là khi $j=1$, đã về đến đầu dãy số thì phải kết thúc.

Bước 4. Tăng $i=4$; nhập số mới $r = 84$; $j = i-1$; $j = 3$, $x[j]=67$.

Lặp lại thủ tục như trên: so sánh $r(=84)$ với $x[j](= 67)$, nếu $(r < x[j])$ là TRUE thì $x[j+1] = x[j]$; $x[j] = r$; nếu FALSE thì $x[j+1] = r$

Kết quả $x[4]=84$

21	33	67	84	
----	----	----	----	--

Bước 5. Tăng $i = 5$, nhập số mới $r = 49$; $j=i-1=4$, $x[j]=84$.

Lặp lại thủ tục như trên, so sánh số mới $r=49$ với $x[j]=84$, đưa 49 về trước 84:

nếu $(r < x[j])$ là TRUE thì $x[j+1] = x[j]$; $x[j] = r$; nếu FALSE thì $x[j+1] = r$. Kết quả:

21	33	67	49	84
----	----	----	----	----

Vì $r = 49$ còn đứng sau 67 nên cần phải sắp xếp tiếp để đưa về vị trí đúng trật tự tăng dần, hiện tại r đang ở vị trí $x[4]$.

Để làm tiếp, phải giảm $j=j-1$; (j mới $= 3$), $x[3] = 67$.

Lặp lại:

So sánh $r(=49)$ với $x[j](= 67)$, nếu $r < x[j]$ TRUE thì $x[j+1] = x[j]$; $x[j] = r$; nếu FALSE thì $x[j+1] = r$

Kết quả:

21	33	49	67	84
----	----	----	----	----

Hiện tại số mới r đang ở vị trí $x[j]=3$, cần phải lặp lại đoạn tính lặp:

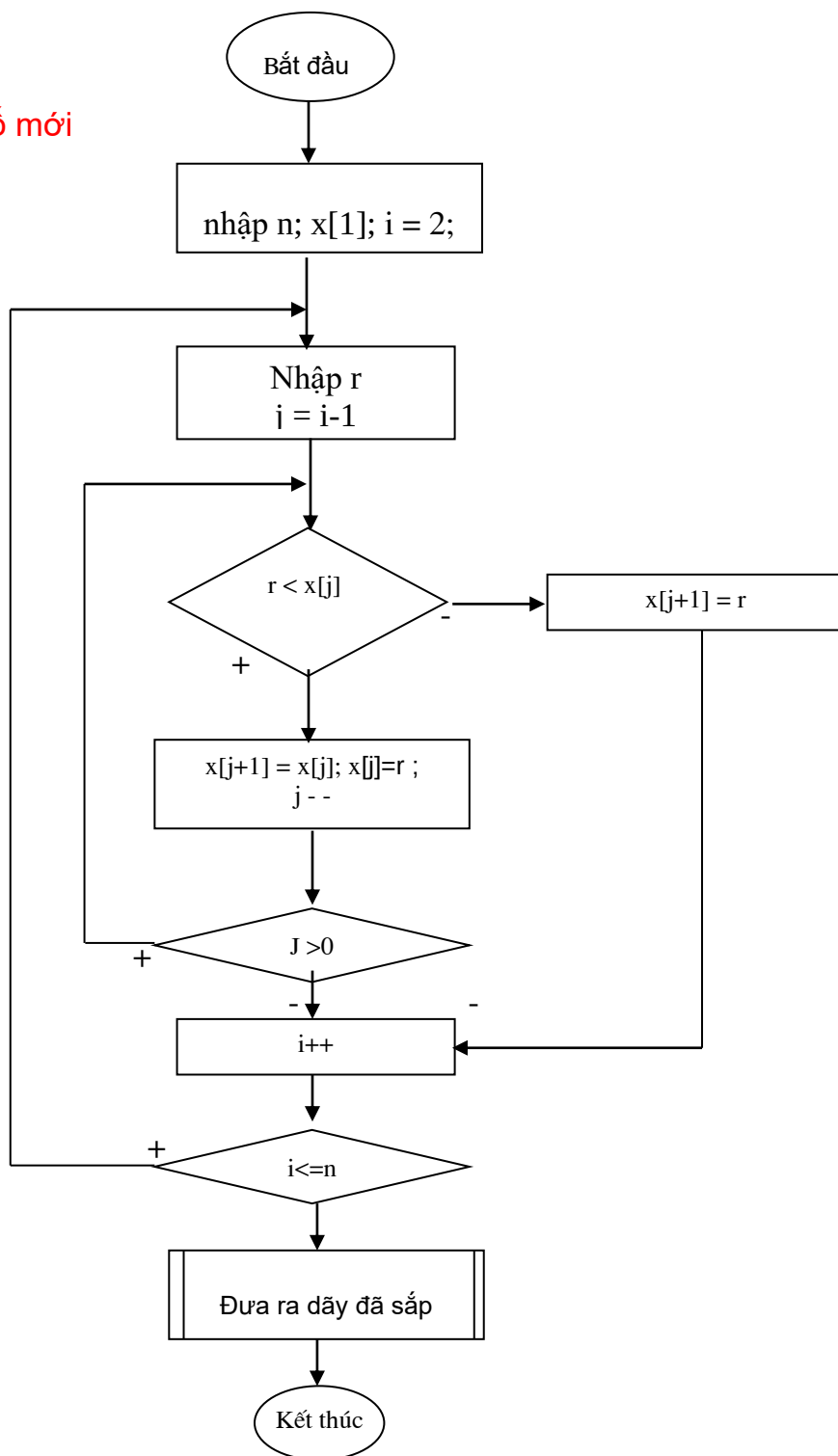
$j=j-1$ (j mới $= 2$), $x[2] = 33$;

So sánh $r(=49)$ với $x[j](= 33)$, nếu $r < x[j]$ TRUE thì $x[j+1] = x[j]$; $x[j] = r$; nếu FALSE thì $x[j+1] = r$. Phép so sánh này cho kết quả FALSE nên sẽ thực hiện $x[j+1] = r$ và thoát ra khỏi bước lặp.

Như vậy, phép giảm $j=j-1$ và so sánh r với $x[j]$: { nếu $r < x[j]$ TRUE thì $x[j+1] = x[j]$; $x[j] = r$; nếu FALSE thì $x[j+1] = r$; } sẽ được lặp cho đến khi $r > x[j]$ hoặc đến khi $j = 1$ thì mới thoát ra để nhập số mới.

Vì bài toán chỉ đặt ra nhập 5 số ($n=5$) nên đến đây sẽ có lệnh kiểm tra kết thúc bước lặp theo i , khi $i=5$ thì thoát ra.

r — số mới



Bài tập về nhà : Viết chương trình C++ sắp xếp chèn dãy số theo giá trị tăng dần theo sơ đồ thuật toán đã cho.

CHƯƠNG 5. CÁC THUẬT TOÁN TÌM KIẾM VÀ ĐỆ QUY

5.1. TÌM KIẾM

5.1.1. Tìm phần tử trong một danh sách tuyến tính ch□a đ□ợc sắp thứ tự.

Tìm kiếm tuyến tính là bắt đầu từ phần tử đầu tiên, tìm một cách tuần tự trong danh sách cho đến khi tìm ra phần tử đã cho hoặc đạt đến cuối danh sách. Kiểu danh sách là mảng các phần tử Kiểu_Ptu.

Ví dụ : Cho danh sách lớp có n người, có các cột mã sinh viên, họ tên, điểm trung bình học kỳ. Vẽ sơ đồ thuật toán và viết chương trình tìm và lập ra danh sách những người có điểm ≥ 8 , lập danh sách những người có điểm cao nhất lớp.

Cách làm :

- Tạo một danh sách lớp - DS1 có n dòng , có các cột

Mã sinh viên	Họ tên	Điểm tb
--------------	--------	---------

- Tạo một danh sách DS2 - những người có điểm ≥ 8 có n dòng , có các cột

Mã sinh viên	Họ tên	Điểm tb
--------------	--------	---------

và tạo danh sách những người có điểm cao nhất lớp – DS3 cũng có n dòng, 2 cột :

Mã sinh viên	Họ tên
--------------	--------

- Nhập số phần tử n;
- Nhập danh sách DS1;
- Tìm điểm lớn nhất Max;
- Tạo DS2 : lần lượt kiểm tra so sánh điểm của từng sinh viên, ai có điểm ≥ 8 thì đưa Mã sinh viên.họ tên, điểm tb học kỳ của người đó sang DS2.
- Tạo DS3 : lần lượt kiểm tra so sánh điểm của từng sinh viên, ai có điểm bằng điểm Max thì đưa Mã sinh viên và họ tên người đó sang DS3.
- Hiện lại DS1, DS2, DS3 .

Hướng dẫn : dùng chỉ số i, j, k lần lượt làm chỉ số trong các danh sách DS1, DS2, DS3.

Khởi gán $i=1, j=0, k=0$.

Đặt tên các biến :

Trong DS1 : masv1[] – mã sinh viên; hoten1[] – họ tên ,tb1[] – điểm trung bình học kỳ,

trong DS2 : masv2[] – mã sinh viên; hoten2[] – họ tên ,tb2[] – điểm trung bình học kỳ,

trong DS3 : masv3[] – mã sinh viên; hoten3[] – họ tên.

Max – điểm lớn nhất.

Phần tạo DS2 :

Cho $j=0$; Cho $i=1$ đến n thực hiện :

Nếu $(tb1[i] \geq 8)$ thì $\{j++; masv2[j]=masv1[i]; hoten2[j]=hoten1[i]; tb2[j]=tb1[i];\}$

Phân tạo DS3 :

Phân tìm điểm Max :

Max = tb1[1];

Cho i=2 đến n thực hiện : nếu (tb1[i]>Max) thì Max=tb1[i];

Cho k=0;

Cho i=1 đến n thực hiện :

Nếu (tb1[i]= Max) thì {k++; masv3[k]=masv1[i];hoten3[k]=hoten1[i];}

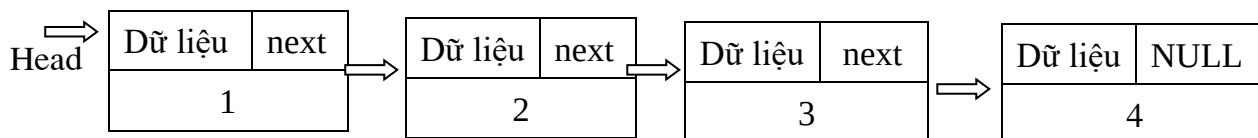
Bài tập :

Cho danh sách lớp có n người, có các cột mã sinh viên, họ tên, điểm trung bình học kỳ. Vẽ sơ đồ thuật toán và viết chương trình tìm và lập ra danh sách những người có điểm ≥ 8 , danh sách những người có điểm cao nhất lớp. Hiện lại các danh sách.

5.1.2.Tìm kiếm trong một danh sách liên kết

Tìm kiếm phần tử trong danh sách liên kết đơn cũng là duyệt danh sách từ đầu cho đến khi tìm thấy phần tử hoặc đến hết danh sách (node ==NULL) mà không thấy, nếu tìm thấy, sẽ trả về node đó.

Ta có sơ đồ :



Nhập dữ liệu để tìm nút có dữ liệu = a, sau đó lần lượt tìm các nút, so sánh dữ liệu node.du_lieu với a, nếu bằng(==) thì đưa ra thông báo “đã tìm thấy”, nếu đến hết danh sách không tìm thấy cũng đưa ra tb “không tìm thấy”

Node* TimNut(Node* list, int a)

```
{ Node* node = list;
  while (node != NULL && node->du_lieu != a)
    node = node->next;
  if(node != NULL){
    cout<<" Da tim thay nut co du lieu ="<<a<<" \n";
    return node; }
  cout<<" Khong tim thay nut co du lieu ="<<a<<" \n";
  return NULL;
}
```

Hàm này đã có khi học danh sách liên kết đơn.

Bài tập : từ phần đã được giải thích và xem lại bài học về danh sách liên kết đơn, vẽ lại sơ đồ thuật toán của tìm kiếm phần tử trong danh sách liên kết.

5.1.3. Tìm kiếm nhị phân

1) Ý tưởng

Tìm kiếm nhị phân đã được nghiên cứu từ lâu trong toán học tính toán.

Muốn tìm một phần tử đã biết chắc chắn có trong một tập Δ đã được sắp thứ tự, lần thứ nhất ta chia đôi tập Δ thành A_1, A_2 ; Lúc đầu tìm phần tử trong A_1 , nếu tìm thấy thì kết thúc, nếu không thấy thì chia đôi A_2 thành B_1, B_2 ; tiếp tục tìm phần tử trong B_1 , nếu tìm thấy thì kết thúc, nếu không thấy thì chia đôi B_2 thành C_1, C_2 ... quá trình tiếp tục cho đến khi tìm được phần tử.

Theo cách chia đôi : tập A_i có lực lượng = $1/2 \Delta \rightarrow 1/2^1$

B_i có lực lượng = $1/4 \Delta \rightarrow 1/2^2$

C_i có lực lượng = $1/8 \Delta \rightarrow 1/2^3$

Ở bước thứ n , tập được chia đôi có lực lượng $\rightarrow 1/2^n$

Với $n=10$ thì $2^{10}=1024$, nghĩa là ta đã xét được trong tập Δ có hơn một nghìn phần tử.

2) Ứng dụng tìm phần tử trong một dãy đã được sắp thứ tự

Tìm phần tử trong danh sách $x_1, x_2, \dots, x_n$ được sắp xếp theo thứ tự tăng dần. found sẽ có giá trị đúng khi tìm được và mid(middle – điểm giữa) là vị trí điểm chia đôi của các phần để tìm .

1. found = FALSE; first = 1; last = n ; //first – điểm đầu, last- điểm cuối

2. while ((first <= last) && (! found))

 { mid = (first + last)/2; // = first+(last-first)/2

 if (item < x[mid]) last = mid-1 ; //item – phần tử cần tìm

 else

 { if(item > x[mid]) first = mid+1;

 else found = TRUE

 }

 }

Bài tập : viết chương trình C++ tìm phần tử trong một tập các phần tử đã nhập theo phương pháp chia đôi(nhị phân)

5.2. ĐỆ QUY

Đệ quy là hiện tượng một thủ tục hay một hàm tham trở đến chính nó (một hàm gọi lại chính nó)

Một thủ tục đệ quy hay hàm đệ quy gồm hai phần:

1. **Phần neo** (anchor — cái neo dùng neo thuyền, tàu bè) trong đó tác động của hàm hay thủ tục được đặc tả cho một hay nhiều tham số.
2. **Phần đệ quy**, trong đó tác động cần được thực hiện cho giá trị hiện thời của các tham số được định nghĩa bằng các tác động hay giá trị được định nghĩa trước đó.

Ví dụ 1: để tính giai thừa có thể mô tả đệ quy :

ở đây phần **neo** là $0! = 1$

đệ quy là với $n > 0$ thì $n! = n * (n-1)!$

Kết quả được tính ngược và cho giá trị cuối cùng

$$n! = n * (n-1)!$$

$$(n-1)! = (n-1) * (n-2)!$$

$$(n-2)! = (n-2) * (n-3)!$$

.....

$$3! = 3 * (3-1)!$$

$$2! = 2 * (2-1)! = 2 * 1!$$

$$1! = 1 * (1-1)! = 1 * 0!$$

$0! = 1$ cuối cùng đi đến phần neo.

Viết hàm đệ quy sau :

```
int giaithua(int n)
{
    if(n==0)
    {
        return 1;
    }
    return n*giaithua(n-1);
}
```

ví dụ 2: số fibonacci

1

1

2

3

với $n > 2$ mỗi số sau là tổng của hai số trước nó

5

8

Neo { $\text{Fib}(1)=1$
 $\text{Fib}(2)=1$

Đệ quy : với $n > 2$, $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

```
int Fib(int n)
{
    if(n <= 2)
    {
        return 1;
    }
    return (Fib(n-1) + Fib(n-2));
}
```

Chương trình :

```

#include <iostream>
using namespace std;

int giaithua(int n)
{
    if(n==0)
    {
        return 1 ;
    }
    return n*giaithua(n-1);
}

int Fib(int n)
{
    if(n<=2)
    {
        return 1;
    }
    return (Fib(n-1)+ Fib(n-2));
}

int main()
{
    cout << " \n\n    DE QUY - RECURSIVE \n\n" << endl;
    int n;
    //float hesoNewton;
    cout<<"    Nhap n :";cin>>n;
    cout<<"\n    Giai thua cua "<<n<<"! = "<<giaithua(n);
    cout<<"\n\n";
    cout<<"\n    So fibonaci cua "<<n<<" = "<<Fib(n);
    cout<<"\n\n";
    system("pause");

    return 0;
}

```

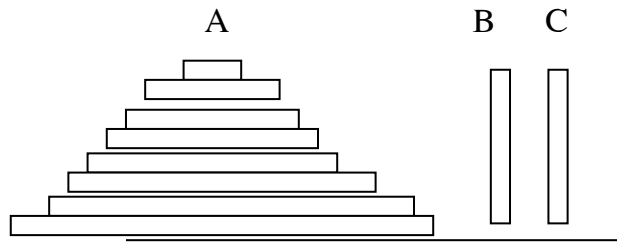
Bài tập

Dùng hàm đệ quy giaithua(int n) viết chương trình C++ tính hệ số nhị thức

$$C_{nk}^n = \frac{N!}{K!(N-K)!}$$

Ví dụ về đệ quy

Tháp Hà nội



Cần phải chuyển các đĩa kích thước khác nhau từ cọc bên trái sang cọc bên phải, với các luật sau:

1. Khi di chuyển một đĩa, nó phải được đặt vào một trong ba cọc đã cho
2. Mỗi lần chỉ di chuyển một đĩa và phải là đĩa ở trên cùng
3. Đĩa lớn hơn không được phép nằm trên đĩa nhỏ hơn. Cho N là số đĩa.

Ta làm đệ quy để chứng minh quy nạp bài toán có lời giải :

Nếu $n=1$, việc chuyển đĩa từ $A \rightarrow C$ là giải được.

- 1, Giả sử đã chuyển được đối với $n-1$ đĩa trên cùng ở $A \rightarrow B$
2. Chuyển đĩa thứ $n \rightarrow$ cột C (C là cọc phụ)
3. Chuyển $n-1$ đĩa từ $B \rightarrow C$, dùng A là cọc phụ:

Chương trình :

```
#include <iostream>
using namespace std;
```

```
void Move(int n, char Dau, char Phu, char Cuoi) //Cọc Đầu, cọc Phụ, cọc Cuối
{
    if(n==1)cout<<"    Chuyen dia tu coc "<<Dau<<" sang coc "<<Cuoi;
    else
    {
        Move(n-1,Dau,Cuoi,Phu);cout<<"\n";
        Move(1,Dau,' ',Cuoi);cout<<"\n";
        Move(n-1,Phu,Dau,Cuoi);cout<<"\n";
    }
}
```

```
int main()
{ cout << "\n\n    THAP HA NOI \n\n" << endl;
  const char coc1 ='A',coc2 ='B',coc3 ='C';
  int sodia;// số đĩa
  cout<<"    Nhập số đĩa : ";cin>>sodia;
  Move(sodia,coc1,coc2,coc3);
  system("pause");
  return 0;
}
```

Kết quả chạy chương trình với số đĩa =2,3,4 (sinh viên tự lấy các đĩa ra thực nghiệm):

```
THAP HA NOI

Nhập số đĩa : 2
Chuyen đĩa từ cọc A sang cọc B
Chuyen đĩa từ cọc A sang cọc C
Chuyen đĩa từ cọc B sang cọc C
Press any key to continue . . .
```

```
THAP HA NOI

Nhập số đĩa : 3
Chuyen đĩa từ cọc A sang cọc C
Chuyen đĩa từ cọc A sang cọc B
Chuyen đĩa từ cọc C sang cọc B

Chuyen đĩa từ cọc A sang cọc C
Chuyen đĩa từ cọc B sang cọc A
Chuyen đĩa từ cọc B sang cọc C
Chuyen đĩa từ cọc A sang cọc C

Press any key to continue . . .
```

```
Nhập số đĩa : 4
Chuyen đĩa từ cọc A sang cọc B
Chuyen đĩa từ cọc A sang cọc C
Chuyen đĩa từ cọc B sang cọc C

Chuyen đĩa từ cọc A sang cọc B
Chuyen đĩa từ cọc C sang cọc A
Chuyen đĩa từ cọc C sang cọc B
Chuyen đĩa từ cọc A sang cọc B

Chuyen đĩa từ cọc A sang cọc C
Chuyen đĩa từ cọc B sang cọc C
Chuyen đĩa từ cọc B sang cọc A
Chuyen đĩa từ cọc C sang cọc A

Chuyen đĩa từ cọc B sang cọc C
Chuyen đĩa từ cọc A sang cọc B
Chuyen đĩa từ cọc A sang cọc C
Chuyen đĩa từ cọc B sang cọc C

Press any key to continue . . .
```